

TTIC 31180 — Probabilistic Graphical Models

Problem Set #3

Due Date: May 7, 2026

1 Kullback-Leibler Divergence [20pts]

- (a) **[5pts]** Consider a factored approximation $Q(X, Y) = Q(X)Q(Y)$ to a joint distribution $P(X, Y)$. Show that minimizing $D(P\|Q)$ (the M-projection) would result in setting $Q(X) = P(X)$ and $Q(Y) = P(Y)$, i.e., the optimal approximation is a product of the true marginals under $P(X, Y)$.
- (b) **[5pts]** Consider the problem of approximating an unknown distribution $P(X)$ with a parameterized distribution $Q(X; \theta)$, where θ is a parameter of the distribution. Suppose that we have access to a set of M I.I.D. samples $\mathcal{D} = \{X^{(1)}, X^{(2)}, \dots, X^{(M)}\}$ drawn from $P(X)$, i.e., $X^{(i)} \sim P(X)$. Maximum likelihood estimation seeks the setting of the parameter θ^* that maximizes the likelihood of the data

$$\theta^* = \arg \max_{\theta} \prod_{i=1}^M Q(X^{(i)}; \theta)$$

Show that this is equivalent to solving for M-projection, i.e., $Q^* = \arg \min D(P\|Q)$.

- (c) **[10pts]** Consider the following form of the joint distribution $P(X, Y)$:

		X			
		1	2	3	4
Y	1	1/8	1/8	0	0
	2	1/8	1/8	0	0
	3	0	0	1/4	0
	4	0	0	0	1/4

Show that the I-projection for $P(X, Y)$ with $Q(X, Y) = Q(X)Q(Y)$ has three distinct minima. Identify these minima and evaluate $D(Q\|P)$ for each of them. What is the value of $D(Q\|P)$ if we set $Q(X, Y) = P(X)P(Y)$?

2 Approximating the Marginal Polytope [10pts]

The local consistency polytope is an approximation to the marginal polytope. This question asks you to investigate the local consistency polytope.

- (a) **[5pts]** Show that for any clique tree, the local consistency polytope is equal to the marginal polytope. You need not prove this from scratch, and are allowed to cite any theorems from the textbook. **Hint:** Think about how you would go about showing that two sets are equal.
- (b) **[5pts]** Show that for some cluster graph that is *not* a tree, the marginal polytope is *strictly* contained in the local consistency polytope. In other words, provide an example of a parametrization of a graphical model that satisfies the local consistency constraints, but does not correspond to any valid probability distribution.

3 Belief Propagation and Bethe Free Energy [10pts]

The fixed points of belief propagation correspond to the minimum stationary points of the Bethe approximation of the free energy (Eqn. 11.19 in Koller & Friedman) for a factor graph. The belief propagation algorithm attains an approximation to the true distribution P when the graph has cycles.

- (a) **[5pts]** This problem asks you to consider a simple example of a distribution that minimizes the Bethe free energy, but is not even a valid distribution.

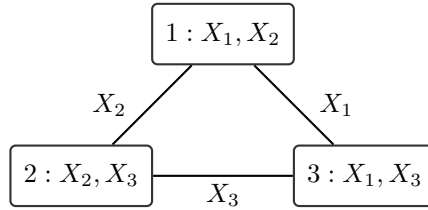


Figure 1: Cluster graph for Problem 3(a).

Consider a simple pairwise Markov random field over three binary variables X_1 , X_2 , and X_3 with the corresponding cluster graph shown in Figure 1. Suppose that the node and sepset beliefs are given by

$$\mu_{1,3}(X_1) = \mu_{1,2}(X_2) = \mu_{2,3}(X_3) = (0.5, 0.5)$$

$$\beta_1(X_1, X_2) = \begin{bmatrix} 0.4 & 0.1 \\ 0.1 & 0.4 \end{bmatrix} \quad \beta_2(X_2, X_3) = \begin{bmatrix} 0.4 & 0.1 \\ 0.1 & 0.4 \end{bmatrix} \quad \beta_3(X_1, X_3) = \begin{bmatrix} 0.1 & 0.4 \\ 0.4 & 0.1 \end{bmatrix}$$

- (i) Show that the set of beliefs above define a set of pseudo-marginal distributions that are calibrated and locally consistent, i.e., that they satisfy the marginalization and normalization conditions (see the description of the “local consistency polytope” in Section 11.3.6 of Koller & Friedman).
- (ii) While the pseudo-marginals are locally consistent, show that they are not marginals of a single joint distribution, i.e., that there can’t be a distribution $P(X_1, X_2, X_3)$ with the above beliefs as its marginals. (Hint: Compute $P(X_1 = 0, X_2 = 0, X_3 = 0), \dots, P(X_1 = 1, X_2 = 1, X_3 = 1)$ and show that they can’t constitute a valid distribution).

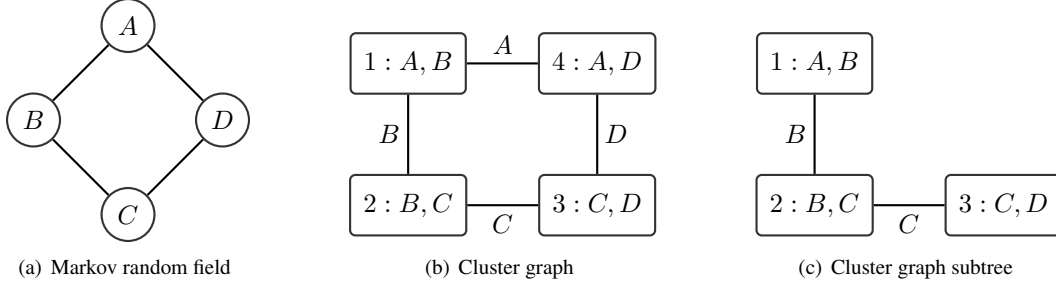


Figure 2: A (a) Markov random field and (b), one choice for the cluster graph, and (c) a potential subtree.

- (b) **[5pts]** Consider a distribution $P_\Phi(A, B, C, D)$ associated with the Markov random field shown in Figure 2(a).

Figure 2(b) depicts one choice for the corresponding cluster graph. As discussed in lecture, belief propagation over the cluster graph results in cluster node and sepset beliefs that are not marginals for the original distribution $P_\Phi(A, B, C, D)$. However, if we remove C_4 , they are valid marginals for the reparameterized distribution $P_\mathcal{T}(A, B, C, D)$ associated with the resulting subtree $\mathcal{T}$ depicted in Figure 2(c).

The cluster tree distribution and the original distribution obey the following relation

$$P_\mathcal{T}(A, B, C, D) = P_\Phi(A, B, C, D) \cdot \frac{\mu_{3,4}(D)\mu_{1,4}(A)}{\beta_4(A, D)}$$

where $r(A, D) = \frac{\mu_{3,4}(D)\mu_{1,4}(A)}{\beta_4(A, D)}$ acts like a *residual* between the joint distribution and the tree distribution.

Show how we can use this residual to bound the error on the estimation of marginals in the cluster graph (i.e., the error between the true marginal $P_\Phi(A, B)$ and the marginal computed from the calibrated graph $\beta_1(A, B)$).



Figure 3: Interactive foreground/background segmentation of an input image.

4 Image Segmentation with Loopy Belief Propagation [35pts]

In lecture, we used image processing tasks to motivate the need for approximate inference strategies. One such task is foreground/background estimation as a specific form of K -ary segmentation, where the goal is to label each pixel in an image as belonging to the foreground or background. This problem asks you to implement loopy belief propagation to perform semi-supervised foreground/background segmentation based on user-provided examples of each class. For example, suppose that Carlos (a famous machine learning researcher) wants to cut himself out of a picture (Fig.3(a)) so that he can paste it against a different background (maybe a wintery scene given his jacket). Carlos is lazy and the most effort that he is willing to put in is to draw two scribbles on the image (Fig. 3(b)), one labeling some foreground pixels (green) and the other labeling background pixels (blue).

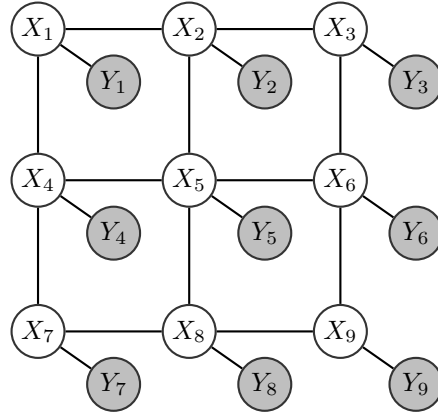


Figure 4: A Markov random field for image segmentation.

This problem can be represented with a pairwise Markov random field $\mathcal{U}$ (Fig. 4), where $X_i \in \mathbf{X}$ is class label for (super)pixel i (i.e., foreground or background), and $Y_i \in \mathbf{Y}$ is an observed feature for that (super)pixel (e.g., it could be the intensity). This gives rise to a joint distribution of the form

$$P(\mathbf{X}, \mathbf{Y}) = \frac{1}{Z} \prod_i \phi_i(X_i, Y_i) \cdot \prod_{(i,j) \in E_{\mathcal{U}}} \phi_{i,j}(X_i, X_j), \quad (1)$$

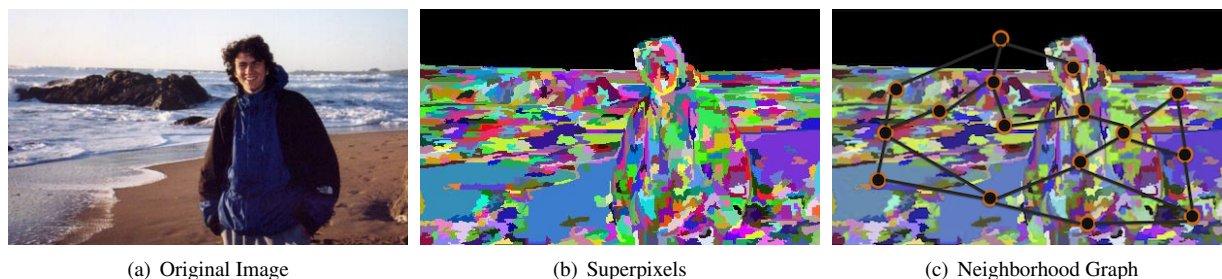


Figure 5: Superpixels and corresponding neighborhood graph.

where the factor $\phi_i(X_i, Y_i)$ relates a pixel's class to its feature representation Y_i , and $\phi_{i,j}(X_i, X_j)$ expresses the influence among neighboring pixels.

A well-known problem with reasoning densely over individual pixels is that the MRFs tend to be quite large (e.g., even a small 300×300 image results in an MRF with 9×10^4 nodes). As we discussed in one of the first few lectures, a common way to deal with this is to generate a more compact MRF over superpixels (Fig. 5(b)), i.e., groups of neighboring pixels that result from an over-segmentation of the image. With this approach, we will modify our problem formulation slightly, letting X_i denote the binary label associated with superpixel i and Y_i be an observed feature of all the pixels assigned to superpixel i . Having performed inference, we then assign the resulting label for superpixel i to all of its corresponding pixels in the original image.

Foreground/Background Segmentation

Your task is to implement semi-supervised foreground/background segmentation via loopy belief propagation. You should provide a function `segment.py` that can be called as

```
$ python segment.py orig-img.png super-map.mat scribble-mask.mat beta
```

where `orig-img.png` is the original image, `super-map.mat` and `scribble-mask.mat` are paths to mat files for the superpixel map and scribble mask (defined below), respectively, and `beta` is a parameter that we will define shortly. This function should save the resulting segmented image as `segmented-img.png`.

Given an image, the foreground/background image sketch, and the superpixels for the image, your implementation will need to (i) generate the features Y_i for each superpixel based upon appearance statistics (we will use Gaussian mixture models); (ii) generate the adjacency matrix for the superpixel image; and (iii) perform loopy belief propagation. The following questions step you through this process.

- (a) **[5pts]** We will define the features Y_i for each superpixel in terms of a separate Gaussian mixture model (GMM) for the foreground and background superpixels. Each mixture model will be defined over the pixel intensities in the CIELUV color space. Note that the original image is in the RGB color space, and you will need to convert it to the CIELUV color space. The problem set includes a `Rgb2Luv.py` class that will do this conversion for you.

The problem set includes a scribble mask (`carlos-beach-scribble-mask.mat`) that is the same size as the original image, where a value of 1 indicates that the corresponding pixel in the original image lies in the foreground, 2 if it is part of the background, and 0 otherwise.

Use the foreground and background pixels in the original image to learn a Gaussian mixture model over CIELUV intensities for each class with 5 components per mixture. You may find `scikit-learn` ([link](#)) useful, particularly the `sklearn.mixture.GaussianMixture` ([link](#)¹) class, which provides a `fit()` function that fits the model ([link](#)¹). If you use `sklearn.mixture.GaussianMixture`, you may want to set `covariance_type` to `diag`, but the defaults for the other parameters are fine.

Reproducibility: Please fix the random seed (e.g., by setting `random_state`) to make your results reproducible.

What to turn in: Report the mean vectors and diagonal of the covariance matrices (as two 3×5 tables each) for the foreground and background GMMs. The `segment.py` file that is provided with the problem set defines a `prettyPrintArray` function that, as you might guess, prints a more aesthetically pleasing version of the array.

- (b) **[5pts]** Next, we will generate the superpixel graph structure. The problem set includes a superpixel map file (`carlos-beach-sp.mat`, variable `labels`) that assigns a superpixel index to each pixel in the original image. Using this map, generate an adjacency matrix for the superpixel neighborhood graph. If the image is divided into S superpixels, this matrix will be of size $S \times S$, and will contain a 1 whenever two superpixels are adjacent (and 0 otherwise).

Hint: Consider scanning rows and columns of the superpixel map to find transitions.

What to turn in: Provide a visualization of the graph associated with the adjacency matrix, overlaid on top of the image.

- (c) **[20pts]** The features at superpixels Y_i will be the average color (CIELUV vectors) of the pixels associated each superpixel. The node potentials will be defined as the likelihood of these feature vectors under the foreground and background GMMs from above,

$$\begin{aligned}\phi_i(X_i = \text{fg}, Y_i) &= P(Y_i | \text{GMM}_{\text{fg}}) \\ \phi_i(X_i = \text{bg}, Y_i) &= P(Y_i | \text{GMM}_{\text{bg}})\end{aligned}$$

Note that `sklearn.mixture.GaussianMixture` provides a `score_samples` function that returns the log probability of a given sample under the associated mixture, which you can take the exponential of to get the probability.

Meanwhile, the edge potential will be defined as

$$\phi_{i,j}(X_i, X_j) = \exp(-\beta \times \mathbb{1}(X_i \neq X_j))$$

where $\mathbb{1}$ is an indicator function.

Generate a cluster graph using these factors according to the adjacency matrix. Feel free to use the structure depicted in Figure 11.6 of Koller & Friedman, which defines a node for each unary and pairwise potential.

Implement loopy belief propagation (see Algorithm 11.1 in Koller & Friedman for a sketch) to calibrate the cluster graph with $\beta = 2$. Stop running loopy belief propagation once the maximum absolute difference between an old and new message is less than 10^{-5} (i.e., use this as a indication that the graph is calibrated). Remember to initialize each message

¹ If you get a 404 error, try changing `%23` in the URL to `#`.

to 1 and to normalize them. **Hint:** compute messages in log-space to avoid numerical instabilities. (You do not need to dampen messages to ensure convergence.)

What to turn in: In addition to your code (see below), an image showing the result with $\beta = 2$.

- (d) **[5pts]** Vary $\beta \in [0, 10]$ with a step size of 2 and propagate superpixel labels back to their constituent pixel segmentations. Why is $\beta = 0$ special? What would happen as $\beta \rightarrow \infty$? What behavior do you see as β increases?

What to turn in: Include the resulting images (e.g., assigning 1 (or 128) to foreground pixels, 2 (or 255) to background pixels, and 0 to all others) for each setting of β (make it clear which setting each corresponds to).

What is included: An archive file (`ps3-segmentation.tar`) that contains the original image (`carlos-beach.png`), segmentation map (`carlos-beach-sp.mat`), scribble mask (`carlos-beach-scribble-mask.mat`), an initial template for `segment.py`, and a Python file (`Rgb2Luv.py`) that converts RGB to CIELUV.

What to turn in: In addition to the items requested above, provide all the code necessary to run your `segment.py` via the command-line call given earlier.

5 Time and Collaboration Accounting

- (a) **[1pt]** Did you work with anyone on this problem set? If so, who?
- (b) **[1pt]** How many hours did you spend on this problem set?
- (c) **[1pt]** Did you use a language model or other form of generative AI (e.g., ChatGPT, Claude, Gemini) to assist with any part of this problem set? If so, identify the specific tool(s) you used and briefly describe how you used them.